

Course Introduction

Welcome to Python Programming

- ✔ **Video:** Introduction to Programming 4 min
- ✔ **Video:** Why Python? 2 min
- ✔ **Reading:** Visual Studio Code 5 min
- ✔ **Reading:** Installing Python paths (Optional for Windows Users) 5 min
- ✔ **Reading:** Installing Python paths (Optional for Mac users) 5 min
- ✔ **Reading:** Required dependencies 5 min
- ✔ **Video:** Environment check for Windows 2 min
- ⌚ **Video:** Environment check for Mac 3 min
- ✔ **Video:** Running code - Command line VS IDE 3 min
- ✔ **Video:** Python syntax, spaces matter 3 min
- ✔ **Reading:** Python syntax cheat sheet
- ✔ **Reading:** Commenting code 5 min
- ✔ **Video:** Variables 6 min
- ✔ **Video:** Basic data types 6 min
- ✔ **Video:** Strings 5 min
- ✔ **Reading:** Basic Data type and Function Cheatsheet 10 min
- ⌚ **Video:** Type casting 2 min
- ⌚ **Video:** User input, console output 8 min
- ⌚ **Reading:** Type casting, a deeper look 10 min
- ⌚ **Programming Assignment:** Type casting input 3h
- 📖 **Practice Quiz:** Knowledge check - Welcome to Python Programming 5 questions
- 📖 **Reading:** Additional resources 5 min

Control flow and conditionals

Basic Data type and Function Cheatsheet

Here's a quick reference for data types in Python.

Data types

Data type	Meaning	Example
string	Text	'Hello', 'Testing 123'
integer	Numbers	-5, 4, 3, 2, 0
float	Decimals	2.4, 5.2, 1000.00

Flow Control

Comparison operators

Operator	Meaning	Example
==	Equals	a == b
!=	Not Equal	a != b
<	Less than	a < b
>	Greater than	a > b
<=	Less than or Equal to	a <= b
>=	Greater than or Equal to	a >= b

Comments

Single-line comments

Placing a # symbol in front of the text you want to be a comment causes Python to ignore everything from that point until the end of the current line.

```
1 # Single Line comment
2
```

Run

Reset

No Output

Multi-line comments

Python does not really have a method for multi line comments, so a # symbol can be used on every line.

```
1 # This is a multiline comment
2 # which can be used for long comments
3
```

Run

Reset

No Output

Inline/code comments

The # symbol will cause Python to ignore everything from that point until the end of the current line, so inline comments can be created in this way.

```
1 x = 1 # assigns value of 1 to x
2
```

Run

Reset

No Output

Built-in Functions

print()

This function looks for the default output device, your terminal, and displays the value passed to it.

```
1 print("Hello")
2
```

Run

Reset

Hello

input()

This function looks for the default input device, your keyboard, and captures the value. This value can then be assigned or used.

```
1 print("Where do you live?")
2 location = input()
3 print("So you live in " + location)
```

Run

Reset

Error on line 2:
location = input()
EOFError: EOF when reading a line

len()

This function returns the length or the count of the elements contained within the structure it is applied on. This may be a string, array, list, tuple, dictionary or any sequence.

```
1 len("Hello")
2 5
```

Run

Reset

str()

This function can be used to convert the provided value into a string

```
1 str(55)
2 '55'
3
```

Run

Reset

No Output

int()

This function can be used to convert the provided value into an int

```
1 int('75')
2 75
```

Run

Reset

75

float()

This function can be used to convert the provided value into a float

```
1 some_int = 10
2 float(some_int)
3 10.0
4
```

Run

Reset

10.0

Creating Functions

Functions in Python require a keyword to define them : **def** followed by an identifier (a name) this forms the function signature. The body of the function contains the code to run when the function is called.

```
1
2 def say_hello():
3     return "Hello there!"
4
5 # With parameters
6 def say_hello(you):
7     return "Hello " + you
8
```

Run

Reset

✔ Completed Go to next item

👍 Like 🗨 Dislike 📄 Report an issue